

01_speckit_init

HXC-159 Phase 1a — GitHub SpecKit initialization in helix_code

Field	Value
Revision	2
Created	2026-07-29
Last modified	2026-07-29
Status	Complete — SpecKit initialized, zero conflicts, verified. Rev 2 adds §9 Bridge-architecture conformance (mid-flight operator requirement).
Workable item	HXC-159 (Task, Queued, High)
Track / branch	T1 / main
Repo HEAD at init	0a4df699b32df9e0eb74c54aee08c52dd90c5765

Table of contents

- [1. Outcome](#)
- [2. Pre-flight: interface discovery](#)
- [3. Pre-flight: conflict proof via throwaway dry-run](#)
- [4. The real init](#)
- [5. Post-init verification — nothing pre-existing was touched](#)
- [6. What was installed](#)
- [7. Findings and risks carried into later phases](#)
- [8. Honest boundary](#)
- [9. Bridge-architecture conformance \(Rev 2\)](#)

1. Outcome

SpecKit is initialized in this repository. `specify init . --integration claude --script sh --force` exited 0, installed 27 files across `.specify/` (18) and `.claude/skills/speckit-*` (10 — see the count note in §6), and modified **zero** pre-existing files.

No fallback was needed. SpecKit is usable here and is available to drive Phases 1b onward via /speckit-specify, /speckit-plan, /speckit-tasks, /speckit-implement.

2. Pre-flight: interface discovery

Per the mandate to work from SpecKit's *actual* interface rather than assumed flags, the CLI was interrogated first.

```
$ which specify
/home/milos/.local/bin/specify
```

```
$ ls -la /home/milos/.local/bin/specify
lrwxrwxrwx 1 milos milos 57 Jul 22 22:32 /home/milos/.local/bin/specify
-> /home/milos/.local/share/uv/tools/specify-cli/bin/specify
```

```
$ specify --version
specify 0.13.5.dev0
```

specify --help reports these subcommands:

Subcommand	Purpose
init	Initialize a new Specify project
check	Check that all required tools are installed
version	Display version and system information
self	Manage the specify CLI itself (check/upgrade, --dry-run)
extension	Manage spec-kit extensions
integration	Manage coding agent integrations
preset	Manage spec-kit presets
bundle	Discover, install, and author Spec Kit bundles
workflow	Manage and run automation workflows

specify init --help documents the exact invocation used, including the in-place form:

```
specify init . --integration claude # Initialize in
current directory
specify init --here --force # Skip
confirmation when current directory not empty
```

Relevant options: `--script {sh,ps,py}`, `--here`, `--force`, `--integration`, `--integration-options`, `--preset`, `--ignore-agent-tools`.

`specify` check confirmed the agent tooling available on this host:

```
|— • Claude Code (available)
|— • Kimi Code (available)
|— • opencode (available)
...
```

`Specify` CLI is ready to use!

Claude Code being *available* is what makes `--integration claude` the correct choice for this session. Note that Gemini CLI, Qwen Code and Codex CLI were reported **not found** on this host — see §7.3.

3. Pre-flight: conflict proof via throwaway dry-run

The task mandates: *“If initialization would overwrite or conflict with anything existing, STOP and report rather than forcing it.”* Since `specify init` has no `--dry-run`, the safe determination was made empirically (§11.4.6 — verify, do not assume): run the identical command into an **empty throwaway directory** in scratch, enumerate exactly what it produces, then test each produced path against the real repo.

```
$ cd <scratch>/speckit_dryrun && specify init . --integration claude --scr
Project ready.
EXIT=0
```

That produced exactly 27 files. Each was then tested for existence in the real repo:

```
=== CONFLICT CHECK: do any speckit paths already exist in real repo? ===
TOTAL CONFLICTS: 0
```

```
=== .claude/skills exists? ===
ls: cannot access '.claude/skills': No such file or directory
```

Zero conflicts. Before `init` the repo’s `.claude/` contained only `settings.json`, `settings.local.json` and `worktrees/` — no `skills/` directory, and no `.specify/` at all. Proceeding was therefore safe, and `--force` served only to skip the interactive “directory not empty” confirmation (the repo has 203 top-level items), not to overwrite anything.

4. The real init

```
$ git status --porcelain | wc -l
27                                     # pre-init dirty count (other strea

$ specify init . --integration claude --script sh --force
Warning: Current directory is not empty (203 items)
--force supplied: skipping confirmation and proceeding with merge

Project          helix_code
Working Path     /home/milos/Factory/projects/tools_and_research/helix_co
Selected coding agent integration: claude
Selected script type: sh

Initialize Specify Project
├─ • Check required tools (ok)
├─ • Select coding agent integration (claude)
├─ • Select script type (sh)
├─ • Install integration (Claude Code)
├─ • Install shared infrastructure (scripts (sh) + templates)
├─ • Ensure scripts executable (5 updated)
├─ • Constitution setup (copied from template)
├─ • Install bundled workflow (speckit installed)
└─ • Finalize (project ready)

Project ready.
SPECIFY_EXIT=0
```

5. Post-init verification — nothing pre-existing was touched

```
$ git status --porcelain | wc -l
28                                     # 27 -> 28

$ git status --porcelain | grep -E '\.specify|\.claude'
?? .specify/
```

The working tree gained **exactly one** status entry, and it is the untracked `.specify/` directory. The delta of +1 with the sole new entry being `?? .specify/` is the proof that `specify init` modified no tracked file. (The 6 M-state entries present both before and after belong to the four other agents live in this checkout — `internal/server/`, `applications/` + `internal/fyneui/`,

submodules/debate_orchestrator, and a research sibling — and are untouched by this work.)

Ignore-status of the two installed trees:

```
$ git check-ignore -v .claude/skills/speckit-specify/SKILL.md
.gitignore:133:.claude/*      .claude/skills/speckit-specify/SKILL.md
```

```
$ git check-ignore -v .specify/memory/constitution.md
NOT IGNORED (trackable)
```

6. What was installed

6.1 .specify/ — 18 files, tracked (this is what gets committed)

```
.specify/init-options.json
.specify/integration.json
.specify/integrations/claude.manifest.json
.specify/integrations/speckit.manifest.json
.specify/memory/constitution.md
.specify/memory/.constitution-template.json
.specify/scripts/bash/check-prerequisites.sh
.specify/scripts/bash/common.sh
.specify/scripts/bash/create-new-feature.sh
.specify/scripts/bash/setup-plan.sh
.specify/scripts/bash/setup-tasks.sh
.specify/templates/checklist-template.md
.specify/templates/constitution-template.md
.specify/templates/plan-template.md
.specify/templates/spec-template.md
.specify/templates/tasks-template.md
.specify/workflows/speckit/workflow.yml
.specify/workflows/workflow-registry.json
```

State recorded by the installer:

```
// .specify/integration.json
{
  "version": "0.13.5.dev0",
  "integration_state_schema": 1,
  "installed_integrations": ["claude"],
  "integration_settings": { "claude": { "script": "sh",
    "invoke_separator": "-" } },
  "integration": "claude",
```

```

    "default_integration": "claude"
}

// .specify/init-options.json
{
  "ai": "claude", "ai_skills": true, "feature_numbering":
    "sequential",
  "here": true, "integration": "claude", "script": "sh",
  "speckit_version": "0.13.5.dev0"
}

```

6.2 .claude/skills/speckit-* — 10 skills, gitignored (see §7.1)

speckit-analyze, speckit-checklist, speckit-clarify, speckit-constitution, speckit-converge, speckit-implement, speckit-plan, speckit-specify, speckit-tasks, speckit-taskstoissues — each a single SKILL.md.

The core five-step flow SpecKit advertises for later phases: /speckit-constitution → /speckit-specify → /speckit-plan → /speckit-tasks → /speckit-implement, plus /speckit-converge; optional /speckit-clarify, /speckit-analyze, /speckit-checklist.

6.3 Not created

specs/ does not yet exist. It is created on first /speckit-specify run via .specify/scripts/bash/create-new-feature.sh. This is expected, not a defect.

7. Findings and risks carried into later phases

7.1 The 10 SpecKit skills are gitignored — they will NOT be committed

.gitignore:133 is .claude/*, with only !.claude/settings.json negated. Every .claude/skills/speckit-*/SKILL.md therefore matches the ignore rule and is invisible to git.

Consequence: on a fresh clone, .specify/ (templates, scripts, workflow, constitution memory) is present but the ten agent-facing skills are **not**. A collaborator must re-run specify init . --integration claude --script sh --force to restore them.

This is a **§11.4.215 concern** (a document that binds work must live tracked in the repository where the work happens) and a §11.4.77 concern (an excluded artifact needs a documented regeneration mechanism). Both are satisfiable cheaply, but the decision is not mine to take unilaterally in this phase, because `.gitignore` is a shared file that other live streams touch:

- **Option A** — add `!.claude/skills/` (and the nested negations git requires) to `.gitignore` so the skills become tracked.
- **Option B** — leave them ignored and register the `specify init` command as the documented §11.4.77 regeneration mechanism in `.gitignore-meta/`.

Recommendation: Option B, because it composes with §7.2 below — `.claude/skills/` is *also* the constitution’s skill-registration target, where entries are symlinks that should not be tracked either. Recorded here for a Phase 2 decision; not actioned.

7.2 `.claude/skills/` is a shared namespace — SpecKit is now a co-tenant

This is directly load-bearing for HXC-159. `constitution/skills/<name>/register.sh` symlinks each constitution skill into `<project>/.claude/skills/<name>` (see 02/03 for detail). SpecKit has now placed 10 *real directories* into that same namespace. They do not collide today (all SpecKit entries are `speckit--`-prefixed; the constitution’s 7 are not), but any future incorporation work that writes into `.claude/skills/` must treat it as shared, and any “clean and re-register” routine must not wipe the SpecKit entries.

7.3 Only the `claude` integration was installed

`specify check` found Gemini CLI, Qwen Code and Codex CLI **not present on this host**, so only `--integration claude` was installed. The repo is governed as a five-carrier, multi-agent project (CLAUDE.md / AGENTS.md / QWEN.md / GEMINI.md / CRUSH.md), and `.specify/integration.json` carries an `installed_integrations` **array**, so additional integrations can be added later with `specify integration` once those CLIs exist on the host. Flagged, not blocking.

7.4 `.specify/memory/constitution.md` is a stock template, not this project’s constitution

The installer reported Constitution setup (copied from template). This file is SpecKit’s own notion of a project constitution and is entirely unrelated to `constitution/Constitution.md` (the HelixConstitution submodule). It must not be confused with, or allowed to shadow, the real governance corpus. Phase 2

should either populate it via `/speckit-constitution` to *point at* the real constitution, or leave it inert — but the distinction must be explicit.

8. Honest boundary

Verified by captured output in this session: the CLI's real interface; that `init` exits 0; that it creates exactly the 27 enumerated files; that it conflicted with nothing; that it modified no tracked file; and the ignore-status of both installed trees.

Not verified in this session: that any `/speckit-*` skill actually *runs* end-to-end and produces a usable spec. Running `/speckit-specify` is Phase 1b work and would create specs/, which is out of scope here. The claim made in §1 is precisely “SpecKit is initialized and its artifacts are on disk” — not “the SpecKit workflow has been exercised.”

9. Bridge-architecture conformance (Rev 2)

A binding operator requirement arrived mid-flight: *“Make sure that SpecKit and Superpowers are used with Bridge extension and with all extensions we have created derived from constitution Submodule!”*

The reference architecture is the **Helix Constitution–Powered SpecKit–Superpowers Bridge**, documented at `constitution/docs/research/extensions/speckit_superpowers/implementation/` (8 documents × 4 formats = 32 files; README.md Revision 1, last modified 2026-07-24).

9.1 The headline answer, stated plainly

The Bridge is a DESIGN, not an INSTALLATION. On this host, exactly one of its seven layers is wired — Layer 2 — and that is the layer I just installed. Layers 3, 4 and 5 do not exist here at all: not as binaries, not as extensions, not as submodules, and not even as the install scripts that the design says would create them.

This is precisely the §11.4.108 SOURCE-vs-RUNTIME gap. The design documents are real, complete and high quality. That is not the same as the layer being available, and this section does not report it as such.

9.2 Layer-by-layer conformance — captured probes

Layer	Bridge design	Present here?	Captured probe
L1 Developer Workstation	CLI agent + context carriers	YES	Claude Code session; CLAUDE.md/ AGENTS.md/ QWEN.md/ GEMINI.md/ CRUSH.md all present at repo root with the §11.4 inheritance blocks
L2 Spec-Kit Core (governance)	specify + skills in .claude/skills/	YES — installed by this task	specify 0.13.5.dev0; 10 speckit-* skills + .specify/ (§4–§6)
L3 SuperSpec bridge (orchestration)	github.com/WangX0111/superspec	NO	command -v superspec → <i>NOT ON PATH</i> ; absent from ~/go/bin, /usr/local/bin
L4 SuperB extension (discipline)	speckit-community.github.io/extensions/superb	NO	specify extension list → “ No extensions installed. ”; .specify/extensions → <i>No such file or directory</i>
L5 SuperBridge MCP (execution)	MCP server, npm-built	NO	command -v superbridge → <i>NOT ON PATH</i> ; grep of .mcp.json + .claude/settings*.json → <i>no superspec/superb/</i>

Layer	Bridge design	Present here?	Captured probe
			<i>superbridge reference</i>
L6 Helix LLM (inference)	Gateway/Brain/ Knowledge/Agents	NO (not on PATH)	command -v helix-llm → <i>NOT ON PATH</i> . Note: a helixllm-gateway.service unit exists in this repo's scripts/systemd/, so L6 may exist as a <i>service</i> rather than a PATH binary — not verified either way , see §9.5
L7 llama.cpp RPC cluster	distributed inference nodes	NO	command -v llama-server rpc-server → <i>NOT ON PATH</i>

Score: 2 of 7 layers present (L1, L2). The Bridge's own self-validation suite advertises "ALL 8/8 CHECKS PASSED — bridge ready"; on this host it would fail at minimum checks 1, 5, 6, 7 and 8.

9.3 The Bridge's own installers do not exist

The design's Quick Start Step 3 and Step 4 name two scripts. Neither is present, and neither is their parent directory:

```
constitution/scripts/extensions/install_speckit_superpowers_bridge.sh  ABS
constitution/scripts/extensions/validate_bridge.sh                      ABS
```

```
$ ls -d constitution/scripts/extensions
ls: cannot access 'constitution/scripts/extensions': No such file or directory
```

The design says that install script would clone spec-kit, superpowers, superspec and superb as submodules under constitution/submodules/. That directory exists but contains none of them:

```
$ ls constitution/submodules/
anti_bluff  clickup_sync  continuum  helix_perf_cache  session_orchestrator
```

```
$ grep -iE 'spec-kit|speckit|superpower|superspec|superb' constitution/.gitmodules
NONE in constitution/.gitmodules
```

So the gap is not “installed but stale” — the installation path itself has never been authored. **Building install_speckit_superpowers_bridge.sh + validate_bridge.sh is unclaimed work** and should become its own tracked item; HXC-159 Phase 2 cannot assume the Bridge and must either depend on that item or scope around it.

9.4 Divergence between the Bridge design and what specify init produced

Honest reconciliation, as required:

#	Divergence	Assessment
1	Skills land in .claude/skills/ — CONFORMS. The design’s install step 4 says “Installs the Spec-Kit extensions into .claude/skills/”. specify init --integration claude did exactly that.	No divergence. The vanilla init produced the same L2 placement the Bridge specifies.
2	Vanilla init installs L2 only. The Bridge expects L2 to be installed <i>by the bridge installer</i> , alongside L3/L4/L5 in one transaction.	Divergence in provenance, not in artifact. The .claude/skills/speckit-* and .specify/ trees are what the Bridge wants at L2; they simply arrived via specify init rather than via the (non-existent) bridge script. When that script is authored it must be idempotent over an already-initialized L2 , not assume a clean slate.
3	No SuperB extension registered. The Bridge routes L2 through L4	Real divergence. specify extension list is empty. SpecKit

#	Divergence	Assessment
	discipline gates (check, brainstorm, implementation-gate, critique, debug, finish).	here runs <i>ungated</i> — the discipline-enforcement layer the Bridge exists to provide is absent. Phase 2 plans built on / speckit-* alone inherit no §11.4 mechanical enforcement from L4.
4	No SuperSpec orchestration. L3 is what actually <i>bridges</i> SpecKit and Superpowers (status, brainstorm, tasks, execute, review).	Real divergence, and the load-bearing one. Without L3 there is no SpecKit ↔ Superpowers bridge at all — the two systems coexist but are not composed.
5	Superpowers is present, but not via the Bridge. The superpowers:* skills (brainstorming, systematic-debugging, test-driven-development, subagent-driven-development, writing-plans, ...) are live in this session. Their source is the Claude Code plugin system (~/.claude-claude1/plugins/installed_plugins.json), not constitution/submodules/superpowers (which does not exist).	Divergence in wiring, not in availability. Superpowers works; it is just not reaching the session through the Bridge's declared path. Any Bridge installer must reconcile with the plugin-sourced copy rather than double-install.
6	.specify/memory/constitution.md is SpecKit's stock template. The Bridge's whole premise is that the <i>Helix</i> Constitution governs the lifecycle.	Real divergence — already flagged independently at §7.4. Under the Bridge this file should point at constitution/Constitution.md, not

#	Divergence	Assessment
		carry generic boilerplate.

9.5 Honest boundary on this section

Verified by captured probe: L3, L4, L5 and L7 are absent from this host; both Bridge install scripts and their parent directory are absent; constitution/submodules/ contains none of the four expected submodules; constitution/.gitmodules has no matching entry; .claude/skills/ placement conforms to the design; superpowers reaches this session via the plugin system.

Explicitly NOT verified: (a) whether **L6 Helix LLM** is running as a *systemd* service on this host — I probed only command `-v helix-llm`, and this repo ships `scripts/systemd/helixllm-gateway.service`, so a service-based L6 is plausible and I did not confirm or refute it; (b) whether L3/L4/L5 exist on any *other* host in the fleet — every probe here is local; (c) I read README.md and IMPLEMENTATION_PLAN.md structurally (headings, architecture diagram, Quick Start, layer table) plus the executive summary — I did **not** read NANO_TASK_ENGINE.md, EXTENSION_DEVELOPMENT.md, TDD_INTEGRATION.md, CONSTITUTION_INTEGRATION.md, SECURITY.md or APPENDIX.md in full, so obligations stated only in those six documents are not reflected here.